LeafGuard: On-Device Plant Disease Detection with a Domain-Adapted Self-Supervised Vision Transformer

Kushagra Dubey
Roll No.: 2300321530109
ABES Engineering College, Ghaziabad, India
Email: kushagra.23b1531207@abes.ac.in

Abstract—Plant disease classifiers trained on the canonical PlantVillage dataset achieve near-saturating accuracy on laboratory imagery yet collapse on real field photographs because of a severe lab-to-field domain gap, and most published systems depend on a server or expose no explanation for their predictions. We present LeafGuard, a production Android application that diagnoses 38 leaf diseases across 14 crop species entirely on-device and explains every prediction with an occlusion-sensitivity saliency map. The classifier is a self-supervised DINOv2 ViT-S/14 backbone with a 38-way linear disease head and a gradient-reversal domain-adversarial head, fine-tuned in three progressively unfrozen stages that combine CutMix/MixUp, consistency regularization, three-fold field oversampling, and a 3-view test-time augmentation. On a held-out 525-image field set the model reaches 86.7% top-1 accuracy—a 19.4 percentage-point improvement over an ImageNet-pretrained Swin-Tiny baseline (67.3%)—while retaining 99.3% on the lab split and 96.1% on a mixed split. This result is the endpoint of a documented seven-iteration development arc in which field accuracy rose from 11% (a PlantVillage-overfit Swin model) through 49%, 34%, and 67.3% to the final 86.7%, each step targeting the specific failure mode of its predecessor. The PyTorch checkpoint is converted to an 86.8 MB FP32 TensorFlow Lite graph in which ImageNet normalization and the final softmax are baked in, and is verified to be bit-identical to the ONNX reference (argmax agreement 3/3, maximum absolute difference 1×10^{-6}). A SHA-256 model-integrity guard and a streaming occlusion heatmap make the deployed system both auditable and explainable without any network dependency.

Index Terms—Plant disease detection, self-supervised learning, vision transformer, domain adaptation, on-device inference, explainable AI, occlusion sensitivity.

I. Introduction

Plant diseases are estimated to destroy between 10% and 40% of global crop yield depending on species and region, and the smallholders who produce the majority of food in many regions rarely have access to a plant pathologist [1], [2]. A smartphone camera that identifies a foliar disease at the moment it is spotted, and tells the grower what to do about it, is therefore a directly useful tool, and the past decade has produced a large body of deep-learning classifiers that approach or exceed human accuracy on the canonical PlantVillage benchmark [3]–[5].

Two obstacles, however, keep these systems out of growers' hands. The first is the lab-to-field domain gap: PlantVillage is a laboratory collection of single leaves photographed against a uniform grey background under controlled lighting, and classifiers trained on it routinely report 98%+ on the lab validation split while collapsing to roughly 30–40% on real field photographs that show whole plants, cluttered backgrounds, and variable illumination [6], [17]. The second is deployment: most published pipelines either depend on a remote server or expose no explanation for a prediction, so a grower asked to spray a chemical or remove a plant has only a bare confidence score to act on. Recent mobile efforts [6], [8] and explainable-AI analyses [9], [10], [20] address parts of this gap, but a single system that is field-robust, offline, and explainable has remained elusive.

This paper presents LeafGuard, a production Android application whose shipped classifier (build v3.5.0.0) closes both gaps, and reports it as the endpoint of a deliberate, iterated development process. Our contributions are:

- A documented seven-iteration evolution from a PlantVillage-overfit Swin-Tiny model (11% field accuracy) to the final domain-adapted self-supervised system (86.7% field accuracy), in which each version was a direct response to the documented failure mode of its predecessor. We report the methodology and result of every step so the field-accuracy trajectory is reproducible and the design decisions are auditable.
- A domain-adapted self-supervised backbone: we replace the ImageNet-pretrained Swin-Tiny used in our earlier release with a DINOv2 [12] ViT-S/14 backbone trained with a gradient-reversal domain-adversarial branch [13], and show this combination lifts field accuracy from 67.3% to 86.7%.
- A three-phase progressive-unfreeze recipe (linear-probe, partial unfreeze, full unfreeze) that pairs CutMix/MixUp, consistency regularization, and three-fold field oversampling to attack the domain gap without catastrophic forgetting of the self-supervised pre-training.
- A bit-identical PyTorch→ONNX→TFLite conversion pipeline in which normalization and softmax are baked into the graph, yielding an 86.8 MB model

that runs entirely on-device with a 3-view test-time augmentation.
- A model-agnostic occlusion-sensitivity explainability path that works under the TensorFlow Lite interpreter (which exposes no gradients), plus a SHA-256 runtime integrity guard, making the deployed system auditable and explainable offline.

The remainder of the paper is organized as follows. Section II surveys related work; Section III describes the dataset, model, fine-tune recipe, and deployment pipeline; Section IV reports experimental results, including the version-by-version evolution; Section V discusses limitations and the improvement roadmap; Section VI concludes.

II. Related Work

A. Plant disease classification

Early deep-learning plant disease classifiers were convolutional networks trained on PlantVillage [1], [2]; subsequent work established that vision transformers [14] and their hierarchical variants [15] match or exceed CNN accuracy on the lab benchmark. Murugavalli and Gopi [3] proposed a vision-transformer pipeline for precision agriculture reporting 99.8%+ accuracy, and Nishankar et al. [4] showed that ConvNeXt-Small and Swin-Small variants are effective for tomato leaf disease recognition. Arshad et al. [5] recently introduced ConvGeM-next, an efficient Swin-based framework for plant disease detection. These works report very high accuracy, but they are evaluated predominantly on the lab distribution and say little about real field imagery.

B. The lab-to-field domain gap

The lab-to-field gap is now a recognized research problem in its own right. Sharif et al. [6] explicitly observe that models trained on PlantVillage perform poorly on real field images and propose a federated multimodal edge-learning method to improve generalization. Salman et al. [17] report that a plain Vision Transformer drops sharply when trained on a PlantVillage subset and tested on the in-the-wild PlantDoc set, and recover roughly 20% of the lost accuracy with a mixture-of-experts ViT, reaching 68% cross-domain accuracy—a useful benchmark for our field number. Jeon et al. [7] tackle the same gap with unsupervised domain adaptation. Prasad and Agniraj [16] combine DANN [13] with CORAL for cross-domain plant disease detection, and Islam et al. [19] recently proposed a domain-adapted lightweight MobileNet ensemble for few-shot plant disease classification that stays near 40 MB for mobile deployment. Our work is closest in spirit to these but differs in three respects: we use a self-supervised rather than ImageNet-supervised backbone, we pair domain adaptation with a progressive-unfreeze schedule that preserves the pre-training, and we report results on an explicit field validation split rather than only on a mixed or augmented test set.

C. Self-supervised features for agriculture

DINOv2 [12] learns general-purpose visual features without labels and is markedly more robust to distribution shift than ImageNet-supervised pre-training. Rithiga and Nadesh [11] recently integrated self-supervised learning with vision transformers for an explainable plant disease framework, and Bai et al. [18] built DINOV2-FCS on the DINOv2-B backbone for joint fruit-leaf disease classification and severity segmentation, reaching 99.67% classification accuracy and demonstrating strong cross-disease generalization. These results motivate the use of self-supervised backbones in this domain. We build directly on DINOv2 ViT-S/14 and quantify the field-accuracy benefit of the self-supervised backbone over an ImageNet-Swin baseline.

D. On-device deployment and explainability

Mobile plant disease applications increasingly target on-device inference via TensorFlow Lite [6], [8]. For explainability, Grad-CAM and its variants are the dominant gradient-based methods [10], [20], but Natarajan et al. [9] compare gradient-based and occlusion-based explainable-AI techniques for plant diseases and find occlusion methods to be a robust, model-agnostic alternative. This is consequential for on-device deployment: the TFLite interpreter exposes neither intermediate activations nor gradients, so Grad-CAM is unavailable, whereas occlusion sensitivity requires only the forward pass. We adopt occlusion sensitivity and detail its on-device implementation.

III. Methodology

A. Dataset

The fine-tune pool deliberately mixes the lab distribution with real field imagery. Four sources totaling 183,004 images were gathered; after deduplication and integrity checking, 90,973 images were retained (92,031 duplicates removed, zero corrupt). The composition is given in Table I. Validation sets are split by domain so that field performance can be measured independently of lab performance: valid_mixed (2,045 images), valid_lab (1,520), and valid_field (525). The training set is 20,600 images (15,200 lab + 5,400 field), with field imagery oversampled three-fold during the partial-unfreeze phase to counter the lab-heavy distribution. The 38 classes span 14 crop species (apple, blueberry, cherry, corn, grape, orange, peach, pepper, potato, raspberry, soybean, squash, strawberry, tomato) covering both diseased and healthy states.

B. Model architecture

The backbone is the self-supervised DINOv2 ViT-S/14 model (facebook/dinov2-small), chosen over an ImageNet-supervised backbone because self-supervised patch tokens encode general visual structure—leaf texture, lesion shape,

TABLE I
Data sources pooled for fine-tuning (raw counts before dedup).

Source	Domain	Images
PlantVillage (Augmented)	Lab	175,734
PlantDoc	Field	2,922
field_corn	Field	4,188
field_alexzcheny	Field	160
Total gathered	—	183,004
Kept after dedup	—	90,973

background context—without over-specializing to ImageNet object categories, and are therefore more stable under the PlantVillage-to-field domain shift. The backbone produces a 384-dimensional feature z from a 224×224 RGB input. Two heads share this feature (Fig. 1):

- A disease classifier $h_c(z) = \text{softmax}(W_c z + b_c) \in \mathbb{R}^{38}$ trained with cross-entropy $\mathcal{L}_{cls}$.
- A domain classifier $h_d(z)$ that predicts lab versus field, connected to the backbone through a gradient-reversal layer (GRL) [13]. The GRL leaves the forward pass unchanged but negates the gradient during backpropagation by a factor λ_d, so the backbone is pushed to produce features that the domain classifier cannot distinguish—i.e., domain-invariant features.

The total training objective is

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_d \, \mathcal{L}_{dom} + \lambda_c \, \mathcal{L}_{cons}, \tag{1}$$

where $\mathcal{L}_{cons}$ is a consistency-regularization term that penalizes disagreement between the prediction on an image and on an augmented version of itself, with weight $\lambda_c = 0.2$ during the partial-unfreeze phase and applied only to field images. The domain weight λ_d follows a schedule (0.1 in linear-probe, 0.3 in partial-unfreeze, 0.1 in full-unfreeze) so the adversarial signal is strongest while the last blocks are learning and is eased during the final low-learning-rate polish. The model has 22.1 M parameters.

C. Progressive unfreezing

Rather than fine-tuning all 22.1 M parameters at once—which catastrophically forgets the self-supervised pre-training—the model is trained in three stages that progressively release the backbone (Table II). Phase 1 learns a good head on frozen features. Phase 2 unfreezes the last four ViT blocks and applies the heaviest regularization (CutMix/MixUp with mix probability 0.5, consistency regularization, three-fold field oversampling, and the strongest domain weight). Phase 3 unfreezes the entire network at a very low learning rate for a final polish. Batch size is 32 throughout and the input resolution is 224×224. A discriminative (layer-wise) learning-rate decay of 0.85 is applied across the twelve ViT blocks in Phase 2 and 0.9 in Phase 3, so deeper blocks learn faster than shallow ones.

TABLE II
Three-phase progressive-unfreeze schedule.

Phase	Epochs	Base LR
1. Linear-probe	3	1×10^{-3}
2. Partial unfreeze	8	1×10^{-4}
3. Full unfreeze	4	5×10^{-6}

TABLE III
Phase details: unfrozen parameters and regularization.

Unfrozen	Params	Regularization
Heads only	39,400	DANN $\alpha = 0.1$
Last 4 blk + heads	7.14 M	CutMix 0.5; consistency 0.2; field 3×; $\lambda_d = 0.3$
All blocks	≈22.1 M	$\lambda_d = 0.1$ (low-LR polish)

D. Test-time augmentation

At evaluation—and at inference on-device—the model is run on three views of each image and the softmaxes are averaged:

$$\bar{p}(x) = \frac{1}{3} \sum_i p(t_i(x)), \quad t \in \{\text{orig, hflip, scale}_{0.875}\}. \tag{2}$$

This is the exact recipe used during validation, so the on-device field number tracks the validated number rather than a single-crop approximation. The heatmap path (Section III-G) deliberately uses a separate single-pass inference so that test-time augmentation does not triple the cost of the occlusion sweep.

E. Conversion and on-device deployment

The PyTorch checkpoint is converted to a TensorFlow Lite graph via the chain PyTorch → ONNX → TFLite (through onnx2tf). The ONNX export uses opset 17 with the legacy exporter (dynamo=False) because the torch 2.9+ dynamo exporter lowers DINOv2's window-attention to an ONNX Loop op that onnx2tf's flatbuffer path cannot lower. onnx2tf converts NCHW to NHWC with batch fixed to 1, and TFLiteConverter produces a FP32 graph (SELECT_TF_OPS enabled for the ViT ops). Before export, graph surgery inserts an ImageNet mean/std subtraction and a final Softmax as ONNX nodes, so the app feeds raw $[0, 1]$ pixels and reads real probabilities directly. The resulting plant_disease.tflite is 86,845,656 bytes (≈86.85 MB), committed in the repository under GitHub's 100 MB file limit. The domain-adversarial head is dropped at export, so the deployed graph is purely the 38-class classifier.

F. Explainability via occlusion sensitivity

Because the TFLite interpreter exposes neither intermediate activations nor gradients, true Grad-CAM is unavailable on-device; we therefore use occlusion sensitivity [9], which needs only the forward pass. The captured leaf is divided into an $8 \times 8 = 64$-cell grid. A baseline forward

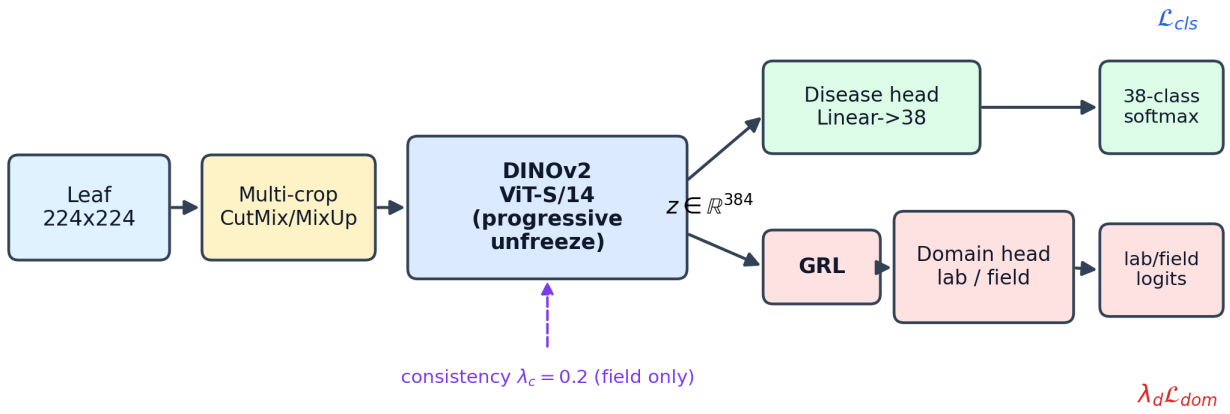


Fig. 1. Fine-tune architecture. A DINOv2 ViT-S/14 backbone feeds a 38-way disease head trained with cross-entropy and an adversarial domain head through a gradient reversal layer. A consistency loss is added during the partial-unfreeze phase and is applied only to field images.

pass records the predicted class confidence $p_c(x)$. Each cell is then occluded in turn—its pixels set to mid-grey $(128, 128, 128)$—and the confidence drop

$$S_{ij} = p_c(x) - p_c(x \odot \mathbf{1}_{\neg ij}) \tag{3}$$

becomes that cell's saliency, yielding 64 occluded passes plus 1 baseline, i.e. 65 forward passes per heatmap. The 64-cell drop field is smoothed with a separable 3×3 Gaussian, sqrt-soft-normalized against the maximum, and color-mapped through a five-stop jet ramp before being upscaled to the image. The heatmap runs on an application-scoped coroutine that survives the scan screen being popped, so the user receives the diagnosis immediately from the single classify pass and the heatmap streams onto the result screen later; a stale-heatmap guard keys each result by scan identifier so a superseded scan's late heatmap cannot clobber the current result.

G. Runtime model integrity

A ModelIntegrity singleton SHA-256-guards the exact .tflite byte buffer the interpreter executes, reading it in 8 KB chunks through a MessageDigest (the buffer is duplicated first so the interpreter's read position is left untouched). On a mismatch the application does not hard-refuse to scan; instead the result screen surfaces a model-integrity warning banner above the diagnosis, so a tampered-model result is read as unverified while the rest of the screen stays usable. The design rationale is that the correct response to a possibly-tampered model is to inform, not block.

IV. Experimental Results

A. Setup

We compare the shipped DINOv2 v3.5 model against the v2.0.0.0 Swin-Tiny ONNX baseline it replaced—an ImageNet-pretrained Swin-Tiny (patch4-window7-224) fine-tuned with CutMix, field oversampling, layer-wise learning-rate decay, and the same 3-view test-time augmentation. Both are evaluated on the same held-out field (525), lab (1,520), and mixed (2,045) validation sets with TTA.

TABLE IV
Top-1 accuracy (%, with 3-view TTA) and footprint.

Model	Field	Lab	Mixed	Size
v2.0 Swin-ONNX	67.3	99.3	94.5	107 MB
v2.5 DINOv2 (ours)	86.7	99.3	96.1	86.8 MB

B. Headline accuracy

Table IV and Fig. 2 report the comparison. The field result is the only number that meaningfully moved: 67.3% → 86.7%, a +19.4 pp gain. Lab accuracy was already saturated at 99.3% (PlantVillage is the training distribution, so this is expected and not a signal of progress), and mixed accuracy rose modestly from 94.5% to 96.1% because the mixed set is dominated by lab images. The honest reading is that the v3.5 methodology fixed exactly the failure it was designed to fix—real-world field photographs—without regressing the lab performance that was already good.

C. Per-class behavior

Across the 38 classes on the mixed split, 21 classes reach precision and recall ≥ 0.95 and 12 reach a perfect 1.0/1.0 (notably the four Grape classes, Corn Common rust, Corn healthy, Peach Bacterial spot, Orange Citrus greening, Squash Powdery mildew, and both Strawberry

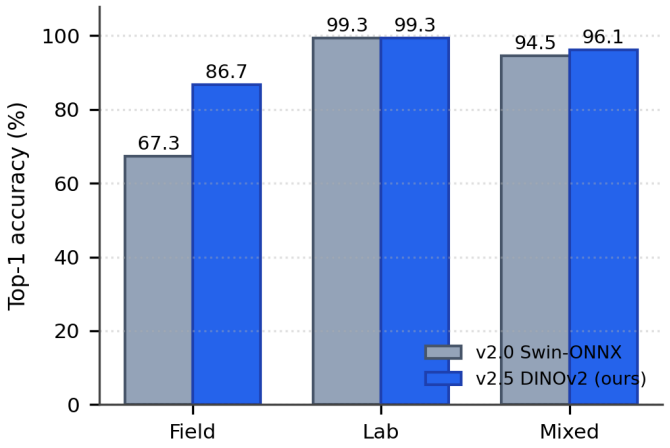


Fig. 2. Field / lab / mixed accuracy: v2.0 Swin-ONNX baseline versus the v2.5 DINOv2 model (3-view TTA). The field split gains 19.4 pp.

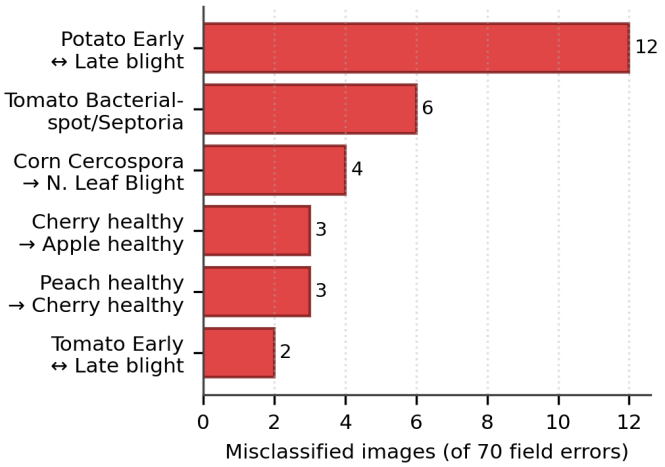


Fig. 3. Top field confusion pairs on the 525-image field validation set. The error is concentrated in within-crop blight confusion.

TABLE V
Weakest and representative strongest classes (mixed split).

Class	Prec.	Rec.	Sup.
Tomato_Bacterial_spot	0.79	0.90	49
Potato_Late_blight	0.81	0.90	48
Potato_Early_blight	0.91	0.78	54
Tomato_Early_blight	0.88	0.92	49
Grape_Black_rot	1.00	1.00	48

classes). The weak classes cluster in two families—the potato blights and the tomato foliage diseases—which are also the top field confusions (Table V). This is consistent with the known difficulty of distinguishing Early- versus Late-blight lesions, which present similar concentric ring patterns.

D. Field confusion analysis

On the 525-image field set the model misclassifies about 70 images (13.3%). The confusions are concentrated, not scattered: the top pairs account for the majority of the errors and are almost all visually-similar diseases within the same crop rather than cross-crop confusions (Fig. 3). The dominant error is Potato Early blight ↔ Late blight (12 images), followed by Corn Cercospora → Northern Leaf Blight (4) and a Tomato Bacterial-spot/Septoria cluster (6). A secondary cluster is healthy-leaf confusion across small-leaf crops (Cherry healthy → Apple healthy, Peach healthy → Cherry healthy), suggesting the model occasionally falls back on leaf shape rather than lesion evidence when no lesion is present. Each of these maps directly to a targeted improvement in Section V.

E. Version-by-version evolution

The 86.7% field result was not obtained in a single training run; it is the endpoint of a documented seven-iteration development arc, each version a direct response to the documented failure of its predecessor. Table VI and Fig. 4 trace the trajectory. The earliest shipped model (v1, a MobileNetV3-Small mobile baseline) was itself an escape hatch from a PlantVillage-overfit Swin model that landed near 36% confidence on diseased tomato and misclassified healthy tomato as strawberry on field photos. The Swin line then went through four iterations on the PlantVillage-overfit base (v1→v2.2): v2.0 pooled 518k PlantVillage derivatives but had zero field coverage and a capping bug that let lab drown field (field ≈11%); v2.1 enforced the lab cap but underfit at 49%; v2.2 introduced a three-stage schedule and effective-number weighting but field accuracy stuck at 34%—the documented PlantVillage→PlantDoc ceiling. The pivot came with v3, which swapped the PlantVillage-overfit base for an ImageNet-pretrained Swin-Tiny and added CutMix/MixUp, three-fold field oversampling, layer-wise LR decay, and four-view TTA, jumping field accuracy to 67.3%. The residual field errors were then same-species disease confusion, which v3.5 targeted by replacing the backbone entirely with self-supervised DINOv2 ViT-S/14 and adding an explicit DANN domain-adversarial head with a gradient-reversal layer, multi-crop consistency regularization, and a three-step progressive unfreeze—reaching the final 86.7%.

F. Bit-identical conversion verification

The shipped TFLite graph was verified against the ONNX Runtime reference on three seeded inputs (random $[0, 1]$, a structured gradient, and a leaf-like 0.3-fill patch with a random 140×140 center). The result is argmax agreement 3/3 and a maximum absolute difference of 1×10^{-6}—well below the 10^{-3} threshold at which softmax ranking could flip, so the on-device model is the validated model. For reference, the retired v2.0 Swin path was verified to argmax 8/8 with a mean maximum absolute difference of 4.42×10^{-8} (ONNX-vs-PyTorch 2.38×10^{-6}).

TABLE VI
Version-by-version evolution: backbone, training recipe, and field/mixed accuracy. Field accuracy rises from 11% to 86.7% across the seven iterations.

Version	Backbone	Key recipe change	Field	Mixed	Diagnosed failure → next step
v1 (mobile)	MobileNetV3-S	FP16 TFLite baseline	—	—	prior Swin overfit on PV
Swin v1	Swin-T (PV)	3 PV sources, SHA dedup	—	98.6%	zero field coverage
Swin v2.0	Swin-T (PV)	add PlantDoc/corn/alex field	11%	84.7%	lab cap bug: lab drowned field
Swin v2.1	Swin-T (PV)	enforce lab/field caps	49%	49%	underfit: 5 epochs too few
Swin v2.2	Swin-T (PV)	3-stage, eff-num weights	34%	66%	PV→PlantDoc ceiling hit
Swin v3	Swin-T (ImageNet)	CutMix/MixUp, 3× field, TTA	67.3%	94.5%	residual blight confusion
v3.5 (ours)	DINOv2 ViT-S/14	+DANN GRL, consistency, 3-step	86.7%	96.1%	— (current)

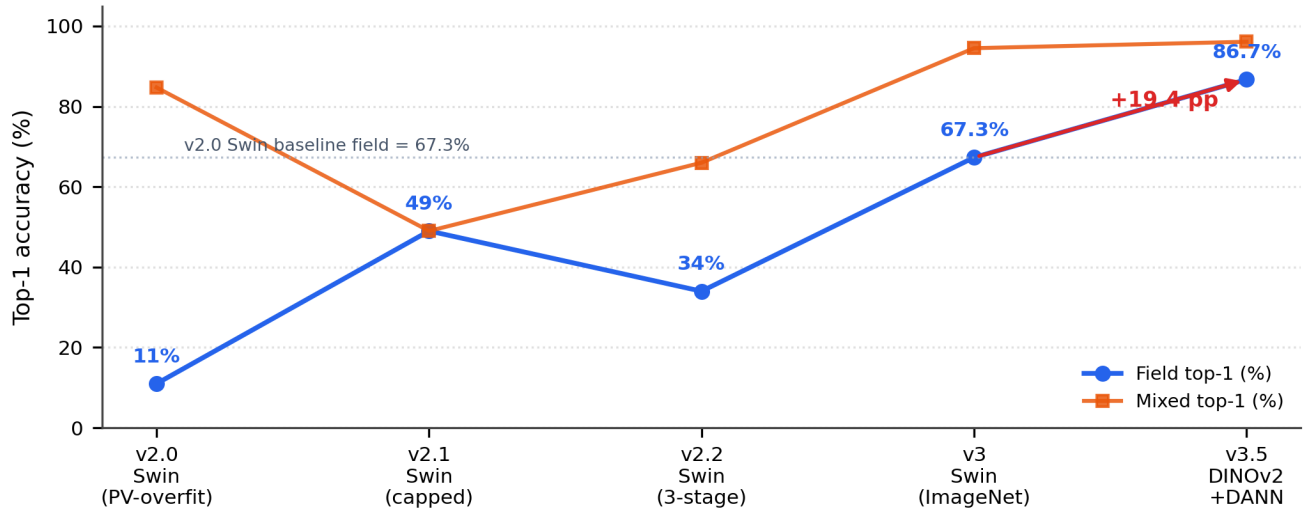


Fig. 4. Field and mixed top-1 accuracy across the seven development iterations. Field accuracy rises from 11% (PlantVillage-overfit Swin) through 49%, 34%, and 67.3% to the final 86.7% (DINOv2 + DANN). The v3→v3.5 step alone adds 19.4 pp.

V. Discussion

A. Why the field gap closed

The +19.4 pp field gain (and the broader 11%→86.7% arc) is attributable to two design choices acting together. The first is the self-supervised backbone: DINOv2 features are trained on a 142 M-image self-supervised objective without labels, so they encode leaf texture, lesion shape, and background context without over-specializing to ImageNet object categories, and stay stable when the domain shifts from a grey-background lab leaf to a cluttered field plant. The second is the domain-adversarial head: by forcing the backbone to produce features the domain classifier cannot label as lab-versus-field, the GRL directly attacks the grey-background-versus-clutter cue that was fooling the v2.0 model. The progressive-unfreeze schedule and the heavy augmentation protect and compound these gains by preserving the pre-training and forcing smooth decision boundaries across domains. The fact that lab accuracy did not regress (99.3% in both versions) indicates the gain came from learning invariance rather than from trading lab accuracy for field accuracy.

B. Limitations

The 525-image field validation set is the honest bottleneck of this study: it is large enough to measure a real field-accuracy signal (each percentage point is approximately five images) but small enough that per-class field numbers are noisy. The field error is also concentrated rather than random, which is both a strength (the failure mode is identifiable) and a limitation (the model has not learned to distinguish a small set of visually-similar blight lesions). Finally, the 86.8 MB FP32 model, while committed under the file limit, is large for an on-device asset, and the occlusion heatmap is slow (≈6–32 s on-device) because of its 65 forward passes.

C. Improvement roadmap and expected results

The system can be improved along four largely independent axes, ranked by leverage. (1) Field data and hard mining (highest leverage): the remaining field error is approximately 12 Potato-blight and 4 Corn-blight confusions; a targeted collection of field Potato and Corn blight photographs combined with focal loss would attack the exact decision boundary that is failing, realistically taking field accuracy from 86.7% toward the 90–92% range. (2) INT8 quantization: full INT8 with a representative field-

image calibration set would cut the 86.8 MB model to ≈22 MB, roughly double inference speed, and typically cost 0–1 pp of accuracy. (3) Per-ABI APK splits: shipping only the matching TFLite native runtime per install removes ≈20 MB of redundant native libraries. (4) Larger backbone and on-device Grad-CAM: a DINOv2-base backbone (≈86 M parameters) would improve field accuracy marginally but pushes the FP32 model to ≈340 MB, viable only after INT8 quantization; an on-device Grad-CAM via a TFLite flex delegate or a custom-op activation tap would yield sharper, gradient-grounded heatmaps at far fewer than 65 passes. Active learning—opt-in upload of low-confidence scans for relabeling and retraining—would let field accuracy drift upward over releases.

VI. Conclusion

We presented LeafGuard, an on-device plant disease detection system whose classifier is a self-supervised DINOv2 ViT-S/14 backbone with a domain-adversarial head, fine-tuned in three progressively unfrozen stages with CutMix/MixUp, consistency regularization, field oversampling, and test-time augmentation. The model reaches 86.7% field accuracy, 99.3% lab accuracy, and 96.1% mixed accuracy on-device—a 19.4 pp field improvement over an ImageNet-pretrained Swin-Tiny baseline, and the endpoint of a documented seven-iteration arc in which field accuracy rose from 11% to 86.7%. The PyTorch checkpoint is converted to an 86.8 MB TFLite graph that is bit-identical to the ONNX reference (argmax 3/3, maximum absolute difference 1×10^{-6}), is SHA-256-guarded at runtime, and is explained by a model-agnostic occlusion-sensitivity heatmap that works under the TFLite interpreter because Grad-CAM is unavailable there. The remaining field error is concentrated in a small set of within-crop blight confusions that targeted field data and focal loss would harden, and INT8 quantization would cut the model to approximately 22 MB. The combination of a field-robust, offline, and explainable classifier makes the system safe to put in a grower's hands.

References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.

[2] D. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of computer vision diagnostics," arXiv preprint arXiv:1511.08060, 2015.

[3] S. Murugavalli and R. Gopi, "Plant leaf disease detection using vision transformers for precision agriculture," Scientific Reports, vol. 15, p. 22361, 2025.

[4] S. Nishankar, V. Pavindran, T. Mithuran, S. Nimishan, S. Thuseethan, and Y. Sebastian, "ViT-RoT: Vision transformer-based robust framework for tomato leaf disease recognition," AgriEngineering, vol. 7, no. 6, p. 185, 2025.

[5] Z. Arshad, A. Javed, and A. Saudagar, "ConvGeM-next: A deep learning framework for plant disease detection," Frontiers in Plant Science, vol. 17, p. 1763739, 2026.

[6] M. Sharif, Y. Zhong, M. Sajid, and F. Marinello, "Real-field-ready and digitally sustainable plant disease recognition via federated multimodal edge learning," Agriculture (MDPI), vol. 16, no. 9, p. 918, 2026.

[7] W. Jeon, T. Kim, S. Choi, and K. Yang, "Bridging the lab-to-field gap in plant disease diagnosis through unsupervised domain adaptation," SSRN Electronic Journal, 2025.

[8] B. R. Reddy et al., "Deep learning based mobile application for automated plant leaf disease detection," IEEE, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11039794

[9] S. Natarajan, P. Chakrabarti, and M. Margala, "Robust diagnosis and meta visualizations of plant diseases through deep neural architecture with explainable AI," Scientific Reports, vol. 14, p. 13695, 2024.

[10] M. Batra and P. Sharma, "Hybrid explainable DeiT-based framework for plant disease classification and severity estimation," Research Square preprint, 2026.

[11] M. Rithiga and R. Nadesh, "SSL VNet: An explainable multimodal hybrid framework integrating self-supervised learning and vision transformers," Frontiers in Artificial Intelligence, vol. 9, p. 1937520, 2026.

[12] M. Oquab et al., "DINOv2: Learning robust visual features without supervision," arXiv preprint arXiv:2304.07193, 2023.

[13] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-adversarial training of neural networks," Journal of Machine Learning Research, vol. 17, pp. 2096–2030, 2016.

[14] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learn. Representations (ICLR), 2021.

[15] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2021, pp. 10012–10022.

[16] P. Prasad and S. Agniraj, "Cross-domain adaptation techniques for robust plant disease detection: A DANN-CORAL hybrid approach," Int. J. Exp. Res. Rev., vol. 42, pp. 68–84, 2024.

[17] Z. Salman, A. Muhammad, and D. Han, "Plant disease classification in the wild using vision transformers and mixture of experts," Frontiers in Plant Science, vol. 16, p. 1522985, 2025. doi: 10.3389/fpls.2025.1522985.

[18] C. Bai, L. Zhang, L. Gao, L. Peng, P. Li, and L. Yang, "DINOV2-FCS: a model for fruit leaf disease classification and severity prediction," Frontiers in Plant Science, vol. 15, p. 1475282, 2024. doi: 10.3389/fpls.2024.1475282.

[19] A. Islam, T. Tahsin, Z. Anjum, M. B. Hasan, and M. H. Kabir, "A domain-adapted lightweight ensemble for resource-efficient few-shot plant disease classification," arXiv preprint arXiv:2512.13428, 2025.

[20] S. M. Alhammad, D. S. Khafaga, W. M. El-hady, F. M. Samy, and K. M. Hosny, "Deep learning and explainable AI for classification of potato leaf diseases," Frontiers in Artificial Intelligence, vol. 7, p. 1449329, 2024. doi: 10.3389/frai.2024.1449329.